

OpenVAS Vulnerability Scanner

Deployment and Scanning Guide

Greenbone Community Edition 24.10
Docker Compose Deployment | Ubuntu 22.04
Authenticated & Unauthenticated Scanning

Date: July 2026

Author: admin

– Description : -

OpenVAS (Open Vulnerability Assessment System) is an open-source, full-featured vulnerability scanner maintained by Greenbone Networks. It is the core engine of the Greenbone Community Edition (GCE), a comprehensive vulnerability management platform that detects security weaknesses across networked systems, applications, and services.

- OpenVAS is deployed via Docker Compose, running the Greenbone Community Edition 24.10 stack as containerised microservices.
- The goal is to perform both unauthenticated and authenticated vulnerability scans against target hosts and produce detailed, actionable security reports.
- Scan results are reviewed through the OpenVAS web dashboard (Greenbone Security Assistant — GSA), accessible via a browser at the host IP address.
- The environment consists of an Ubuntu 22.04 host running Greenbone Community Edition containers communicating over an internal Docker network.

– Environment Setup : -

The environment consists of the following components:

- Ubuntu 22.04 host system (Static IP: 192.168.1.113)
- Docker and Docker Compose installed for container orchestration
- Greenbone Community Edition 24.10 deployed via official Docker Compose manifest
- OpenVAS web interface accessible at <https://192.168.1.113> (port 443)
- Two scan targets: soar system (192.168.1.112) and ubuntu-self (192.168.1.113)
- One authenticated target (ubuntu-self) using SSH credentials for deep package-level scanning
- One unauthenticated target (soar system) for network-level vulnerability discovery

Docker Container Architecture

The Greenbone Community Edition runs as a suite of specialised Docker containers. Each container handles a specific function within the vulnerability management platform:

Container	Function
gvmd	Greenbone Vulnerability Manager — core orchestrator and database interface
openvas-scanner	Scanner engine — executes NVT (Network Vulnerability Test) plugins
osspd-openvas	OSP daemon — bridges gvmd to the OpenVAS scanner
gsad / nginx	Web interface (GSA) served via HTTPS on port 443
pg-gvm	PostgreSQL database storing scan results and configurations
redis-server	In-memory cache for NVT data used by the scanner
vulnerability-tests	NVT feed — 181,000+ vulnerability check plugins
cert-bund-data	German CERT advisory data feed
dfn-cert-data	DFN-CERT advisory data feed
scap-data	SCAP security content data feed
notus-data	Notus scanner data for package-level CVE checks
data-objects	Report formats and GVM configuration objects
gpg-data	GPG keys for feed signature verification

```
kiranpami@kiranpami-Standard-PC-1440FX-P11X-1996:~$ sudo docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	NAMES	CREATED	STATUS	PORT
3257e8d29de8	registry.community.greenbone.net/community/nginx:latest	"/docker-entrypoint..."	greenbone-community-edition-nginx-1	4 weeks ago	Up 26 hours	0.0.
0.0:443->443/tcp, [::]:443->443/tcp, 80/tcp, 0.0.0.0:9392->9392/tcp, [::]:9392->9392/tcp	registry.community.greenbone.net/community/gvm-tools	"/usr/local/bin/entr..."	greenbone-community-edition-gvm-tools-1	4 weeks ago	Created	
831e5a6411cf	registry.community.greenbone.net/community/gsad:stable	"/usr/local/bin/entr..."	greenbone-community-edition-gsad-1	4 weeks ago	Up 26 hours	
77bd013ala86	registry.community.greenbone.net/community/gvmd:stable	"/usr/local/bin/entr..."	greenbone-community-edition-gvmd-1	4 weeks ago	Up 26 hours (healthy)	
1392e459f3a3	registry.community.greenbone.net/community/ospd-openvas:stable	"/usr/bin/tini -- /u..."	greenbone-community-edition-ospd-openvas-1	4 weeks ago	Up 26 hours	
39acdcbf51a7	registry.community.greenbone.net/community/openvas-scanner:stable	"/bin/sh -c 'cat /et..."	greenbone-community-edition-openvas-1	4 weeks ago	Up 9 minutes	
df7bcb1bb9711	registry.community.greenbone.net/community/openvas-scanner:stable	"/usr/local/bin/open..."	greenbone-community-edition-openvasd-1	4 weeks ago	Up 26 hours	
35bda96cbff3	registry.community.greenbone.net/community/pg-gvm:stable	"/usr/local/bin/entr..."	greenbone-community-edition-pg-gvm-1	4 weeks ago	Up 26 hours (healthy)	
cf48545abbd	registry.community.greenbone.net/community/dfn-cert-data	"/bin/init.sh"	greenbone-community-edition-dfn-cert-data-1	4 weeks ago	Up 9 minutes (healthy)	
451e46a0b71b	registry.community.greenbone.net/community/report-formats	"/bin/init.sh"	greenbone-community-edition-report-formats-1	4 weeks ago	Up 8 minutes (healthy)	
093fc9807297	registry.community.greenbone.net/community/openvas-scanner:stable	"/bin/sh -c 'printf ..."	greenbone-community-edition-configure-openvas-1	4 weeks ago	Exited (0) 8 minutes ago	
d356545b6c5	registry.community.greenbone.net/community/redis-server	"docker-entrypoint.s..."	greenbone-community-edition-redis-server-1	4 weeks ago	Up 26 hours	6379
8e915baa5613	registry.community.greenbone.net/community/gvm-config:latest	"/bin/init.sh"	greenbone-community-edition-gvm-config-1	4 weeks ago	Exited (0) 2 minutes ago	
58c5fdd95cc4	registry.community.greenbone.net/community/gpg-data	"/bin/init.sh"	greenbone-community-edition-gpg-data-1	4 weeks ago	Exited (0) About a minute ago	
f0c37869b53d	registry.community.greenbone.net/community/pg-gvm-migrator:stable	"/usr/bin/tini -- /u..."	greenbone-community-edition-pg-gvm-migrator-1	4 weeks ago	Exited (0) About a minute ago	
188ef596bbd3	registry.community.greenbone.net/community/gsa:stable-slim	"/usr/local/bin/entr..."	greenbone-community-edition-gsa-1	4 weeks ago	Up 6 minutes (healthy)	
1db6bae42c59	registry.community.greenbone.net/community/cert-bund-data	"/bin/init.sh"	greenbone-community-edition-cert-bund-data-1	4 weeks ago	Up 5 minutes (healthy)	
712152b036c8	registry.community.greenbone.net/community/vulnerability-tests	"/bin/init.sh"	greenbone-community-edition-vulnerability-tests-1	4 weeks ago	Up 5 minutes (healthy)	
344322f7829d	registry.community.greenbone.net/community/notus-data	"/bin/init.sh"	greenbone-community-edition-notus-data-1	4 weeks ago	Up 2 seconds (health: starting)	
db995e352f7a	registry.community.greenbone.net/community/data-objects	"/bin/init.sh"	greenbone-community-edition-data-objects-1	4 weeks ago	Up 4 minutes (healthy)	
638a54b8a67d	registry.community.greenbone.net/community/scap-data	"/bin/init.sh"	greenbone-community-edition-scap-data-1	4 weeks ago	Up 4 minutes (healthy)	
79b15fa3daf1	registry.community.greenbone.net/community/scap-data	"/bin/init.sh"	greenbone-community-edition-scap-data-1	4 weeks ago	Up 4 minutes (healthy)	

Figure 1 — `sudo docker ps -a` output showing all Greenbone Community Edition containers running on the host.

The output of **sudo docker ps -a** confirms that all core containers are in the "Up" state. Key observations:

- greenbone-community-edition-nginx-1 is Up, listening on ports 443 and 9392 — serving the web interface.
- greenbone-community-edition-gvmd-1 is Up and healthy — the core manager is operational.
- greenbone-community-edition-ospd-openvas-1 is Up — the OSP daemon is ready to accept scan jobs.
- greenbone-community-edition-pg-gvm-1 is Up and healthy — the PostgreSQL database backend is active.
- greenbone-community-edition-redis-server-1 is Up — the NVT caching layer is functional.
- Feed containers (vulnerability-tests, cert-bund-data, notus-data, scap-data) show healthy status after completing their data synchronisation.

– Architecture Diagram : -

The diagram below illustrates the end-to-end scanning workflow for this deployment — from the initial network path down to remediation and verification. It shows how a request travels from the public network through the authenticated SSH session on the Ubuntu target, into the OpenVAS scanning engine, and on through detection, reporting, and remediation.

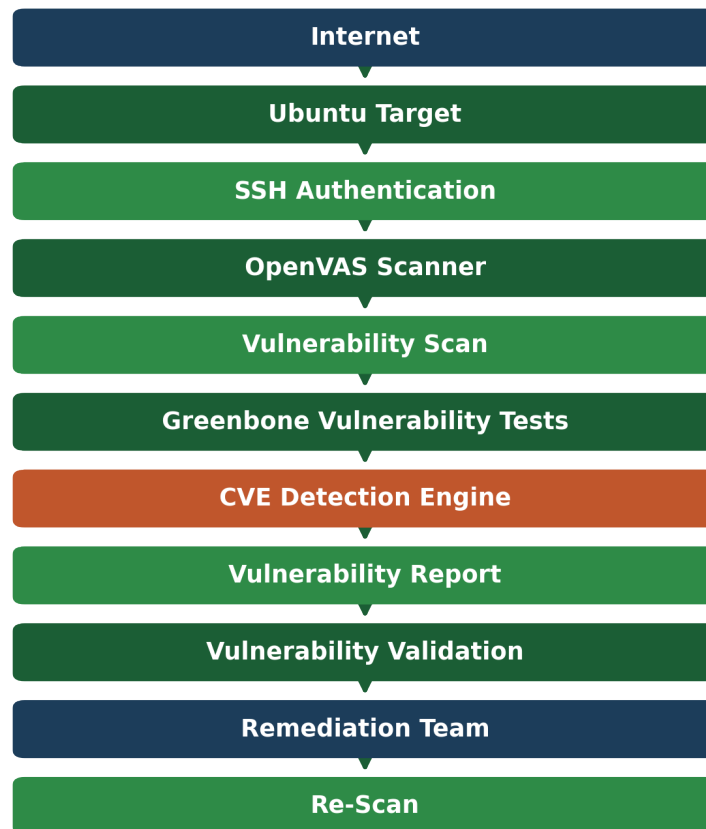


Figure A — End-to-end scanning workflow: from the internet-facing network path through SSH-authenticated scanning on the Ubuntu target, to CVE detection, reporting, validation, remediation, and re-scan.

Key stages in this flow:

- Internet — the external network boundary from which the scan host and its targets are reached.
- Ubuntu Target — the host being assessed (ubuntu-self, 192.168.1.113).
- SSH Authentication — credentials configured in OpenVAS are used to log in to the target for deep, package-level inspection.
- OpenVAS Scanner — the Greenbone scanning engine that executes NVT plugins against the target.
- Vulnerability Scan — the active scan session that probes ports, services, and installed packages.
- Greenbone Vulnerability Tests — the 181,187 NVT plugins from the Community Feed used to check for known issues.
- CVE Detection Engine — matches confirmed findings to their corresponding CVE identifiers and CVSS scores.
- Vulnerability Report — the structured report of results, organised by severity, host, port, and CVE.
- Vulnerability Validation — findings are reviewed to confirm they are genuine and correctly scoped.
- Remediation Team — the engineers who apply patches or configuration fixes based on the report.
- Re-Scan — a follow-up scan that confirms the fix was effective and closes the loop.

– Accessing the OpenVAS Web Interface : -

- After the containers start, the Greenbone Security Assistant (GSA) is accessible at <https://192.168.1.113>.
- The interface uses a self-signed TLS certificate — browsers show a security warning which can be acknowledged to proceed.
- The admin account is used to manage all scan tasks, targets, and reports.
- The login page presents the OpenVAS Community Edition branding with Username and Password fields.

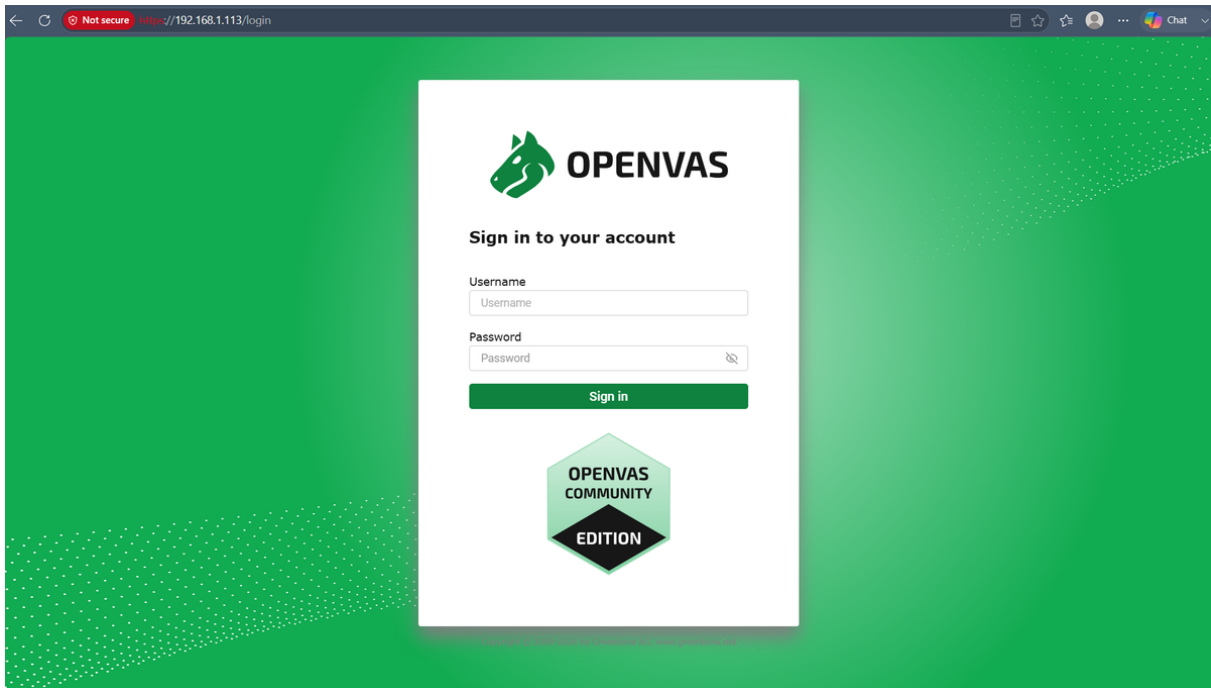


Figure 2 — OpenVAS Community Edition login page accessed at <https://192.168.1.113/login>.

Dashboard Overview

After successful login, the main Dashboard provides a real-time overview of the vulnerability management environment:

- **Tasks by Severity Class** — pie chart showing the distribution of scan tasks by their highest detected severity (Medium, Critical).
- **Tasks by Status** — pie chart showing task completion states (Done, Running).
- **CVEs by Creation Time** — historical chart of CVE volume growth from 1990 to present, showing the rapid acceleration of disclosed vulnerabilities.
- **NVTs by Severity Class** — breakdown of the 181,187 available Network Vulnerability Test plugins: 70,472 Medium, 66,977 High, 34,399 Critical, 5,349 Low.

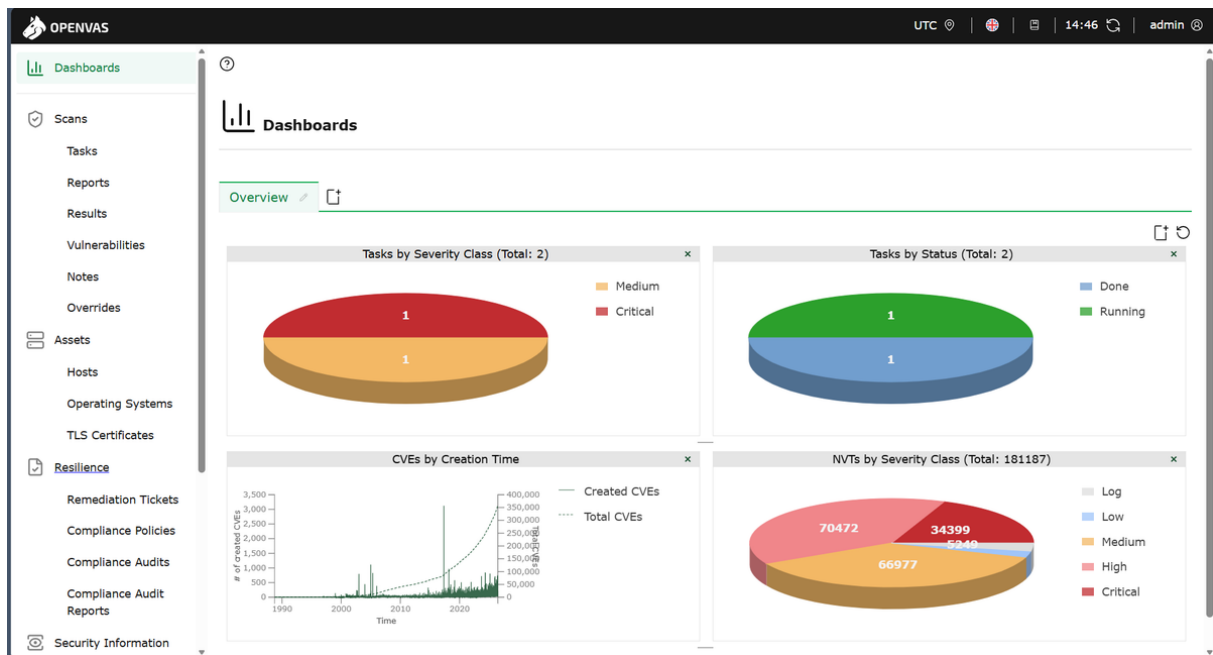


Figure 3 — OpenVAS main Dashboard showing tasks by severity, task status, CVE history, and NVT plugin distribution across 181,187 tests.

The NVT count of **181,187** confirms that the vulnerability feed has been successfully synchronised from the Greenbone Community Feed. This feed is updated regularly and contains detection plugins for known CVEs, configuration weaknesses, SSL/TLS issues, exposed services, and default credentials.

– Configuring Scan Targets : -

Targets are the hosts or IP ranges that OpenVAS will scan. Each target definition includes the host address, port list, and optionally authentication credentials for deeper scanning. Two targets were configured for this deployment:

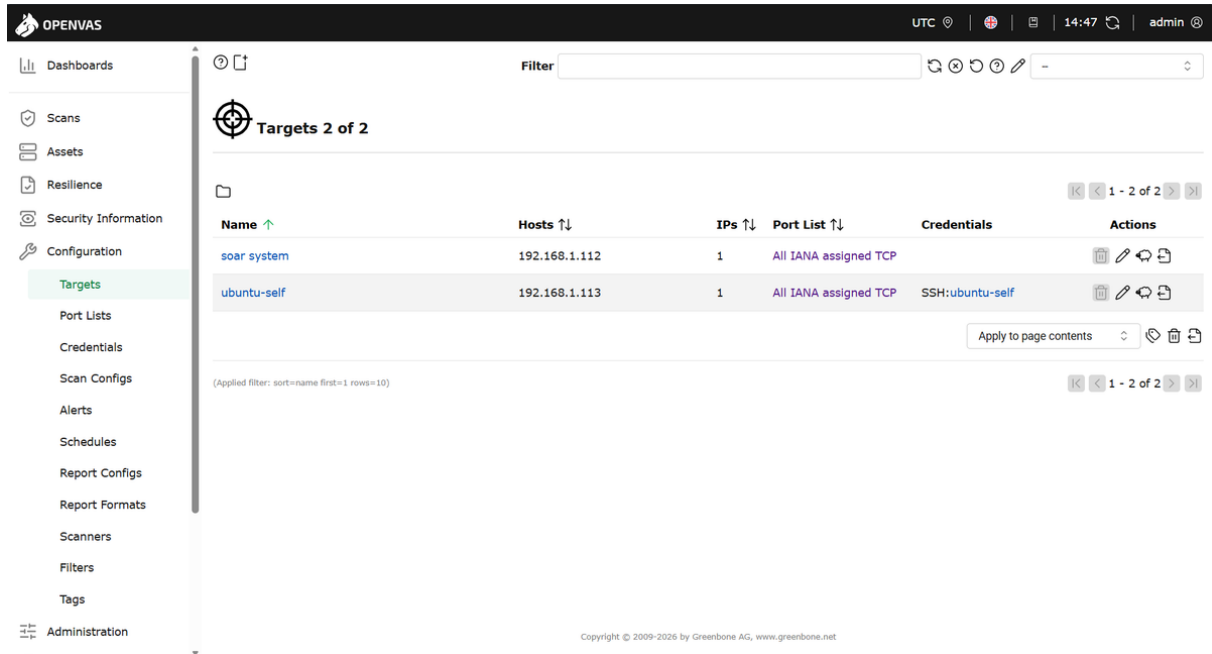


Figure 4 — Targets list showing two configured targets: *soar system* (unauthenticated) and *ubuntu-self* (authenticated via SSH).

Unauthenticated Target — *soar system*

Target Name	soar system
Host IP	192.168.1.112
Port List	All IANA assigned TCP
Credentials	None (Unauthenticated Scan)
Scan Type	Network-level vulnerability discovery — simulates external attacker

Authenticated Target — *ubuntu-self*

Target Name	ubuntu-self
Host IP	192.168.1.113 (the scanner host itself)
Port List	All IANA assigned TCP
Credentials	SSH: ubuntu-self (Username + Password — login: kiranpami)
Scan Type	Deep package-level scanning via authenticated SSH session

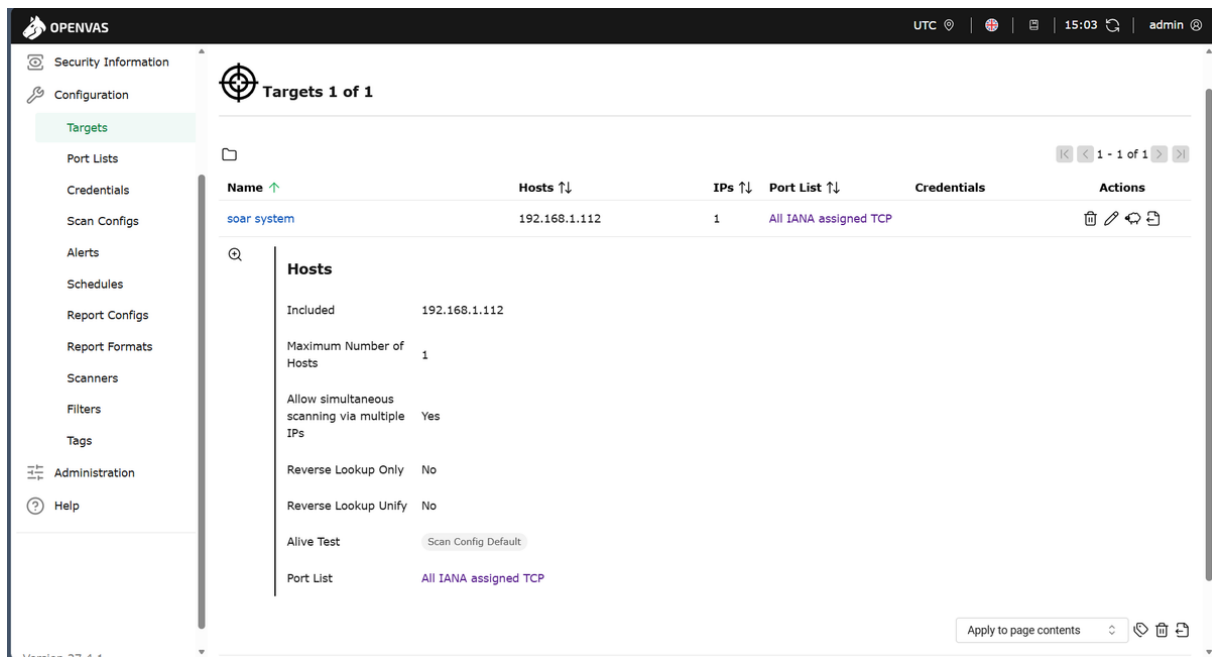


Figure 5 — Target detail for soar system (192.168.1.112): All IANA assigned TCP ports, no credentials — unauthenticated scan.

- Hosts: 192.168.1.112 — single IP target
- Port List: All IANA assigned TCP — covers all registered TCP ports
- Alive Test: Scan Config Default — checks host availability before scanning
- Credentials: None — external black-box scan, no login to the target

– Credential Configuration for Authenticated Scanning : -

For the ubuntu-self target, SSH credentials were created to allow the scanner to log in to the target system and perform deep package inspection. Authenticated scans reveal far more vulnerabilities than unauthenticated scans because the scanner can read installed package versions, check kernel parameters, and inspect configurations directly.

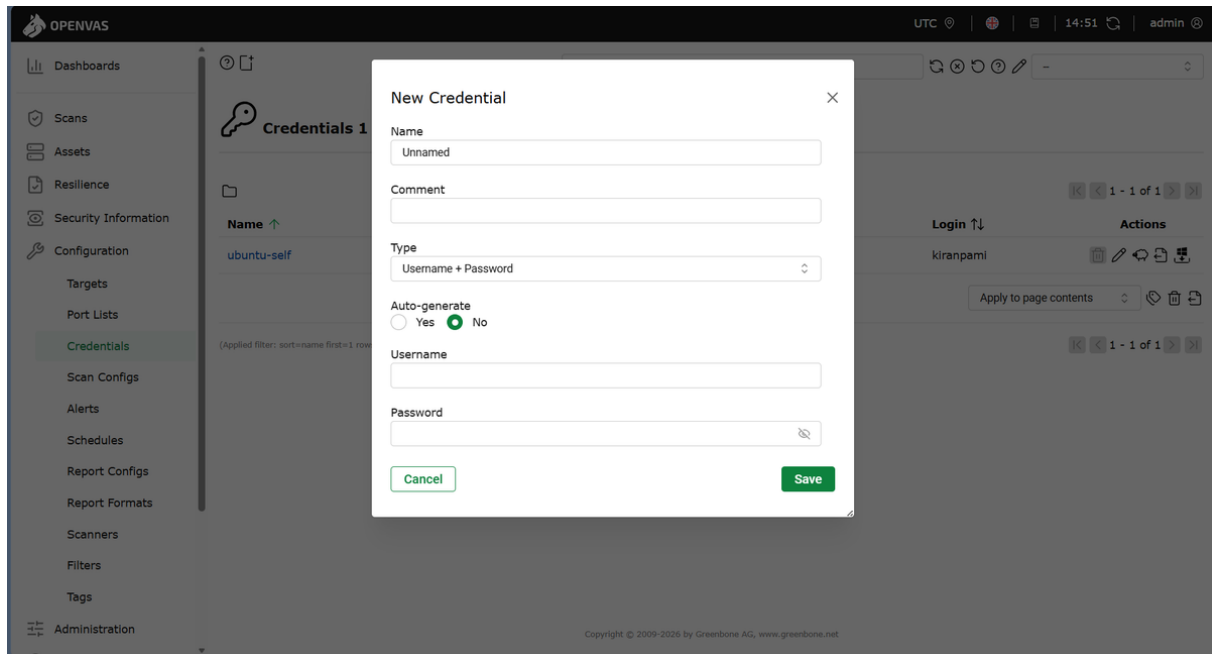


Figure 6 — New Credential dialog: Username + Password type configured for SSH access to ubuntu-self (login: kiranpami).

- **Credential Name:** ubuntu-self
- **Type:** Username + Password — SSH authentication using OS login credentials
- **Auto-generate:** No — credentials manually specified
- **Login:** kiranpami (as shown in the Credentials list)
- The credential is then linked to the ubuntu-self target, enabling the scanner to authenticate via SSH during the scan and inspect the installed package database.

– Creating and Running Scan Tasks : -

Scan tasks combine a target with a scan configuration to define what will be scanned and how. Tasks are created under Scans → Tasks and run asynchronously — status updates from Requested → Running → Done.

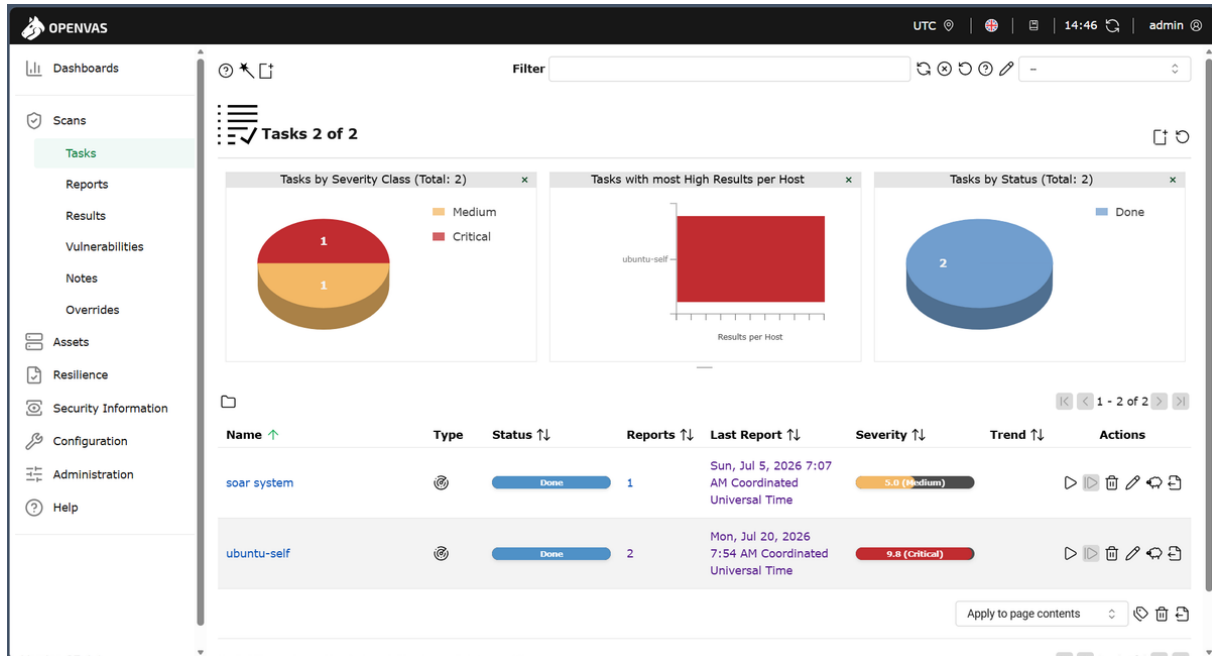


Figure 7 — Scans → Tasks view: two completed tasks — soar system (Medium 5.0) and ubuntu-self (Critical 9.8).

Task Summary

Field	soar system	ubuntu-self
Target IP	192.168.1.112	192.168.1.113
Scan Type	Unauthenticated	Authenticated (SSH)
Status	Done	Done
Reports	1	2
Last Report	Sun Jul 5, 2026 7:07 AM UTC	Mon Jul 20, 2026 7:54 AM UTC
Highest Severity	5.0 (Medium)	9.8 (Critical)

The **Tasks by Severity Class** chart confirms one task reached Critical severity and one reached Medium — clearly illustrating the difference between authenticated and unauthenticated scan depth. Both tasks show **Done** status in the Tasks by Status chart.

The ubuntu-self task has run **twice** (Reports: 2), demonstrating the ability to re-scan targets over time to track remediation progress.

– Reviewing Scan Results : -

Completed tasks generate detailed Reports under Scans → Reports, providing a structured view of all discovered vulnerabilities organised by severity, host, port, and CVE reference.

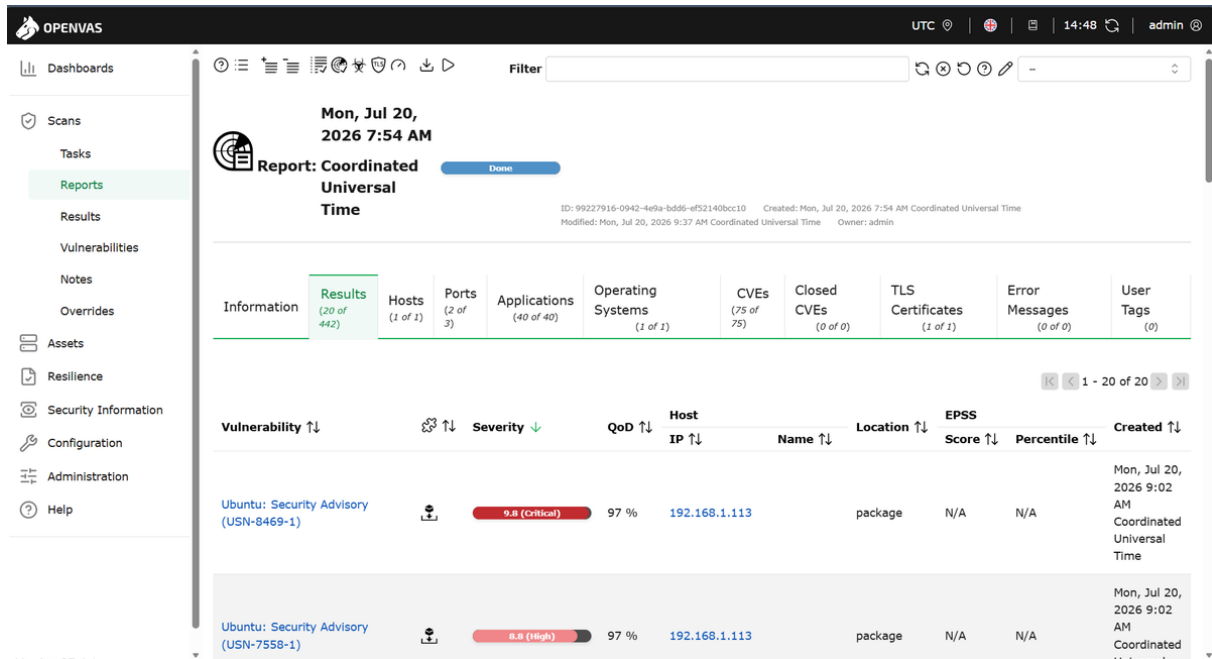


Figure 8 — Report for ubuntu-self (Jul 20, 2026): 442 total results, top findings Critical 9.8 and High 8.8 Ubuntu Security Advisories.

Authenticated Scan Report — ubuntu-self

Report ID	99227916-0942-4e9a-bdd6-ef52140bcc10
Scan Date	Monday, July 20, 2026 — 7:54 AM UTC
Status	Done
Total Results	442 vulnerabilities detected
Hosts Scanned	2 of 1 (host with multiple network interfaces)
Ports Discovered	3 of 3
Applications	40 of 40 applications fingerprinted
Operating Systems	1 of 1 — Ubuntu Linux
CVEs Referenced	75 unique CVEs identified
TLS Certificates	1 certificate analysed

The Results tab displays vulnerabilities sorted by severity (highest first). Top findings:

- **Ubuntu Security Advisory USN-8469-1** — Severity 9.8 (Critical), QoD 97%, host 192.168.1.113, via package inspection
- **Ubuntu Security Advisory USN-7558-1** — Severity 8.8 (High), QoD 97%, host 192.168.1.113, via package inspection

The high QoD (Quality of Detection) of **97%** confirms these are verified findings based on actual package version inspection — not heuristic guesses. Authenticated scanning enables this level of certainty because the scanner reads the installed package database directly via SSH.

Report Tabs Explained

- **Results** — complete vulnerability findings sorted by severity
- **Hosts** — per-host breakdown of findings
- **Ports** — which TCP/UDP ports had vulnerabilities detected
- **Applications** — software and service fingerprints on the target
- **CVEs** — direct cross-reference to the 75 CVEs identified
- **Closed CVEs** — CVEs checked but confirmed NOT vulnerable
- **TLS Certificates** — SSL/TLS certificate analysis for HTTPS services

– Vulnerability Detail Analysis : -

Clicking any vulnerability in the Results list opens its full detail page, providing complete technical information: CVE description, detection methodology, affected software, remediation guidance, and external references.

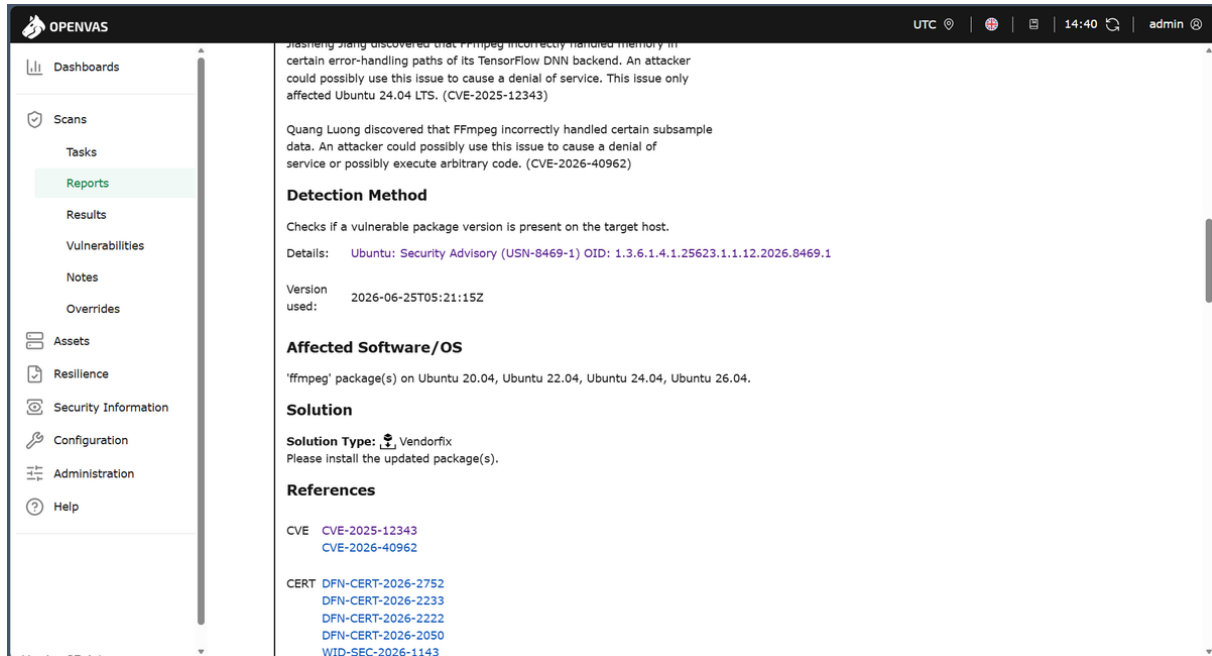


Figure 9 — Vulnerability detail for USN-8469-1: CVE descriptions, detection method (package check), affected software, VendorFix solution, and external references.

Critical Finding: Ubuntu Security Advisory USN-8469-1 (FFmpeg)

CVE-2025-12343 — Jiasheng Jiang discovered that FFmpeg incorrectly handled memory in certain error-handling paths of its TensorFlow DNN backend. An attacker could use this to cause a denial of service. Affects Ubuntu 24.04 LTS.

CVE-2026-40962 — Quang Luong discovered that FFmpeg incorrectly handled certain subsample data. An attacker could use this to cause a denial of service or execute arbitrary code.

Detection Method	Checks if a vulnerable package version is present on the target host
Detection Type	Package version inspection via authenticated SSH session
OID	1.3.6.1.4.1.25623.1.1.12.2026.8469.1
NVT Version	2026-06-25T05:21:15Z
QoD	97% — confirmed via package database lookup
Affected OS	ffmpeg package(s) on Ubuntu 20.04, 22.04, 24.04, 26.04
Solution Type	VendorFix — official patch available from Ubuntu
Action Required	sudo apt update && sudo apt upgrade ffmpeg

References

- CVE-2025-12343, CVE-2026-40962
- DFN-CERT-2026-2752, DFN-CERT-2026-2233, DFN-CERT-2026-2222, DFN-CERT-2026-2050
- WID-SEC-2026-1143

This demonstrates the power of authenticated scanning — the scanner identifies the exact installed package version, matches it against the known-vulnerable range in the NVT database, and provides targeted remediation with vendor patch information.

– Authenticated vs Unauthenticated Scanning : -

A core objective of this deployment was to demonstrate the significant difference in scan depth and result quality between authenticated and unauthenticated scans.

Attribute	Unauthenticated (soar system)	Authenticated (ubuntu-self)
Target IP	192.168.1.112	192.168.1.113
Credentials	None	SSH Username + Password
Scan Depth	Network / service level only	Full system: packages, kernel, configs
Highest Severity	5.0 (Medium)	9.8 (Critical)
Total Results	Fewer — external view only	442 findings
CVEs Found	Limited to exposed services	75 unique CVEs
QoD Score	Lower — heuristic detection	97% — confirmed package match
Detection Method	Port scanning, banner grabbing	Package version database inspection
Use Case	External attacker simulation	Internal compliance & patch assessment

Key observations:

- The authenticated scan discovered 442 vulnerabilities — most software vulnerabilities are invisible to external scanning alone.
- Highest severity jumped from 5.0 (Medium) to 9.8 (Critical) — missing these would create a dangerous false sense of security.
- QoD is significantly higher for authenticated findings — package version checks are definitive with no ambiguity.
- Authenticated scanning is essential for compliance assessments (PCI-DSS, ISO 27001, CIS Benchmarks) where definitive patch status must be reported.

– Understanding Vulnerability Severity : -

OpenVAS uses the Common Vulnerability Scoring System (CVSS) to rate the severity of each finding. The CVSS score (0.0–10.0) reflects the potential impact and ease of exploitation.

Severity	CVSS Score	Description
Critical	9.0 – 10.0	Highly exploitable, remote code execution possible. Immediate action required.
High	7.0 – 8.9	Significant risk. Can lead to privilege escalation or data loss. Patch within days.
Medium	4.0 – 6.9	Moderate risk. Exploitation requires specific conditions. Patch within weeks.
Low	0.1 – 3.9	Minimal risk. Difficult to exploit or limited impact. Regular maintenance.
Log	0.0	Informational only. No vulnerability score — detection or best-practice note.

Quality of Detection (QoD)

QoD is an OpenVAS-specific metric (0–100%) indicating how reliable a detection is:

- **97–100%** — Package version confirmed via authenticated scan. Definitive finding.
- **70–96%** — Active probe response confirmed the vulnerability.
- **30–69%** — Indirect evidence or version-based heuristics. Lower confidence.
- **1–29%** — Best-guess estimate based on service fingerprint only.

The Critical finding (USN-8469-1) has QoD 97% — the scanner directly confirmed the vulnerable ffmpeg package version by reading the installed package database, making this one of the most reliable detection types possible.

– Vulnerability Management Lifecycle : -

Once OpenVAS identifies vulnerabilities, they move through a structured lifecycle that takes a finding from initial discovery to confirmed closure. Authenticated and unauthenticated scanning are run as distinct stages so that both external, black-box exposure and deep, package-level issues are captured before results are consolidated into a single report.

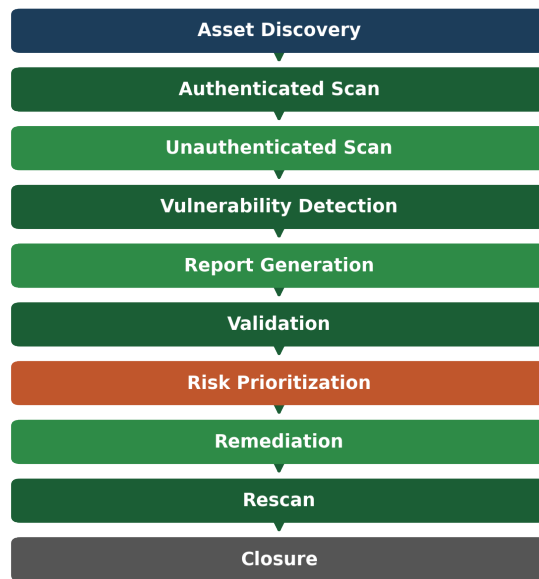


Figure 10 — Vulnerability Management Lifecycle: 10 stages from Asset Discovery through both scan types, detection, reporting, validation, prioritisation, remediation, rescan, and closure.

Lifecycle Stages Explained

1. Asset Discovery

Identify and catalogue the hosts and IP ranges in scope, e.g. soar system (192.168.1.112) and ubuntu-self (192.168.1.113), before any scan is configured.

2. Authenticated Scan

Run a credentialed SSH scan against the target so OpenVAS can inspect installed package versions, kernel parameters, and configuration files directly.

3. Unauthenticated Scan

Run a network-level scan with no credentials to see what an external attacker would find — open ports, exposed services, and banner-based checks.

4. Vulnerability Detection

NVT plugins from the Greenbone Community Feed evaluate the scan data and confirm which known vulnerabilities are present.

5. Report Generation

Confirmed findings are compiled into a structured report with severity, QoD, affected host/port, and CVE references.

6. Validation

The security team checks that each finding is genuine and correctly scoped, filtering out false positives before they are actioned.

7. Risk Prioritization

Validated findings are ranked by CVSS severity, QoD confidence, and asset criticality to decide what gets fixed first.

8. Remediation

The assigned engineer applies the fix — a package upgrade, a configuration change, or another corrective action.

9. Rescan

The affected host is scanned again to confirm the vulnerability no longer appears, providing evidence the fix worked.

10. Closure

Once the rescan confirms resolution, the finding is closed and archived as part of the audit trail.

– CVE vs NVT : Understanding the Difference : -

Two terms appear constantly in OpenVAS reports: CVE and NVT. They are closely related but serve completely different purposes. Understanding the distinction is essential for interpreting scan results correctly.

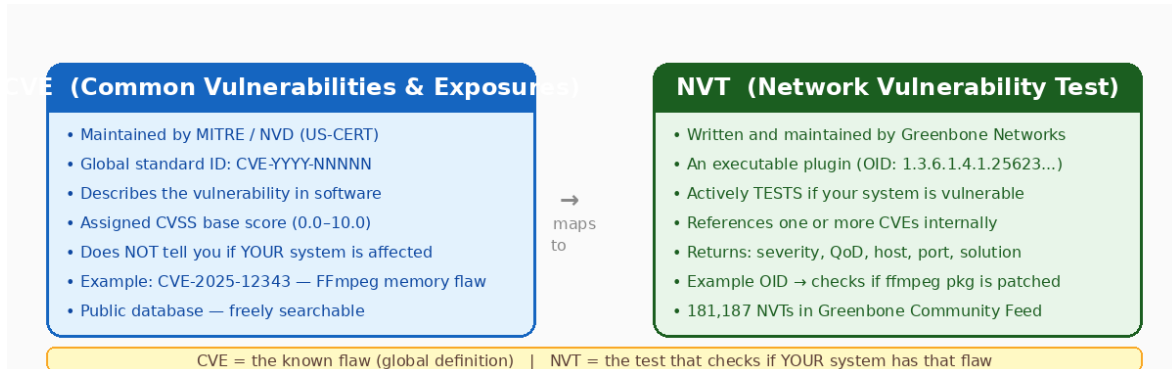


Figure 11 — CVE vs NVT: CVE is the global vulnerability definition; NVT is the test plugin that checks if your system is affected.

CVE — Common Vulnerabilities and Exposures

A CVE is a **globally standardised identifier** for a known security vulnerability in software or hardware. It is maintained by MITRE Corporation and published in the US National Vulnerability Database (NVD).

CVE Property	Details
Maintained by	MITRE Corporation / NVD (US-CERT / CISA)
Format	CVE-YYYY-NNNNN (e.g. CVE-2025-12343)
What it describes	A specific security flaw in a specific software component
CVSS Score	Assigned by NVD — base score 0.0–10.0 (impact + exploitability)
What it does NOT do	It does not test your system. It only describes the flaw.
Who uses it	Vendors, researchers, governments, compliance frameworks, tools
Access	Free public database: https://nvd.nist.gov

Example CVE: CVE-2025-12343 — "FFmpeg incorrectly handles memory in TensorFlow DNN backend error paths, allowing denial of service." Score: 9.8 Critical.

NVT — Network Vulnerability Test

An NVT is an **executable plugin** within OpenVAS that actively tests whether a specific target system is affected by a known vulnerability. It is the "how we check" — the practical implementation of a CVE detection.

NVT Property	Details
Maintained by	Greenbone Networks (Community Feed) + community contributors

NVT Property	Details
Format	OID: 1.3.6.1.4.1.25623.X.X.X.X (unique plugin identifier)
What it does	Actively probes the target: checks package versions, banners, configs
References	Points to one or more CVEs internally
Output	Result: severity, QoD, host, port, description, solution
Feed size	181,187 NVTs in Greenbone Community Feed (updated daily)
QoD (accuracy)	97% for package checks; lower for heuristic banner-based checks

Example NVT: OID 1.3.6.1.4.1.25623.1.1.12.2026.8469.1 — reads the installed ffmpeg package version via SSH, compares to vulnerable range, reports CVE-2025-12343 if unpatched.

Key Differences at a Glance

Attribute	CVE	NVT
What it is	A vulnerability definition / ID	An active test plugin
Created by	MITRE / NVD	Greenbone Networks
Tests your system	No — purely descriptive	Yes — actively probes target
Stored where	nvd.nist.gov (public database)	Inside OpenVAS scanner engine
Relationship	Source reference (root cause)	References 1 or more CVEs
Output	CVE ID, CVSS score, description	Result with host, port, severity, QoD
Example	CVE-2025-12343	OID: 1.3.6.1.4.1.25623.1.1.12.2026.8469.1
Update cycle	Published when vendor discloses	Daily feed updates from Greenbone

How They Work Together in OpenVAS

- Step 1 — A software vendor discovers and patches a flaw → discloses it as CVE-2025-12343 with CVSS 9.8.
- Step 2 — Greenbone engineers write an NVT plugin that checks if the vulnerable package version is installed.
- Step 3 — The NVT is published in the Greenbone Community Feed (daily update).
- Step 4 — OpenVAS runs the NVT against your target via SSH → reads installed ffmpeg version.
- Step 5 — NVT confirms vulnerable version found → creates a Result referencing CVE-2025-12343, CVSS 9.8, QoD 97%.
- Step 6 — Report shows you exactly which host, package, and version is affected — and the fix.

Bottom line: CVEs tell the world a flaw exists. NVTs tell you whether *your* systems are actually affected. OpenVAS bridges the two — it translates global CVE knowledge into targeted, actionable findings on your

specific infrastructure.

– Integration with Ticketing Tools : -

OpenVAS does not exist in isolation. In a production security workflow, scan findings are automatically fed into ticketing and SOAR platforms so that every vulnerability becomes a tracked, assigned, and measurable work item. This section covers the integration architecture, supported platforms, and how the SOAR automation pipeline connects OpenVAS to TheHive 5.

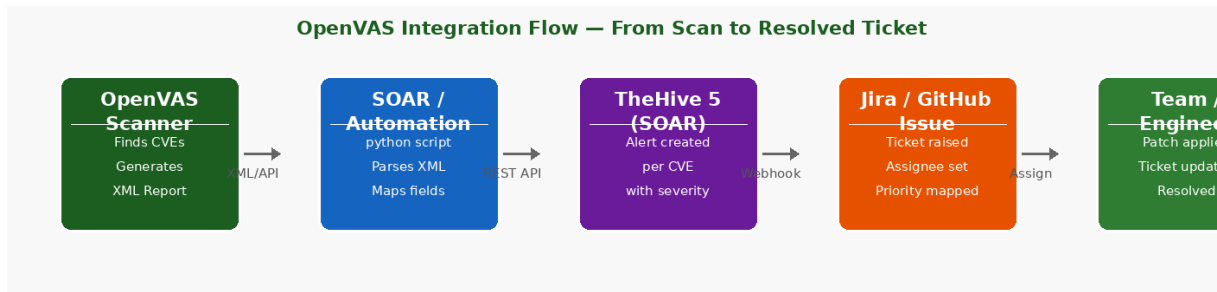


Figure 12 — Integration flow: OpenVAS scan → automation script → TheHive 5 alert → Jira/GitHub ticket → assigned engineer resolves.

Why Integration Matters

- Without integration: findings sit in OpenVAS reports, manually reviewed, easily missed or forgotten.
- With integration: every Critical/High finding automatically becomes a ticket with an owner, due date, and audit trail.
- Reduces mean time to remediation (MTTR) — findings are acted on within minutes, not days.
- Provides compliance evidence — every vulnerability has a documented lifecycle from detection to resolution.
- Enables metrics — track open CVEs by severity, age, team, and asset across the entire organisation.

Supported Integration Platforms

Platform	Type	Integration Method	Best For
TheHive 5	SOAR / Incident Response	REST API (POST /api/v1/alert)	SOC teams, incident management, automated triage
Jira	Issue Tracker	Jira REST API / Atlassian Connect	Dev teams, sprint planning, SLA tracking
GitHub Issues	Issue Tracker	GitHub REST API	Open source projects, dev-sec collaboration
ServiceNow	ITSM Platform	ServiceNow REST API / MID server	Enterprise IT, CMDB integration, change management
PagerDuty	Alerting / On-call	Events API v2	Critical severity alerts, on-call escalation
Slack / Teams	Messaging	Webhook / Bot API	Real-time notifications for security channels

– Conclusion : -

Summary

This deployment successfully demonstrated end-to-end operation of OpenVAS Greenbone Community Edition 24.10:

- Docker Compose deployment: All 18 Greenbone containers deployed and running healthy on Ubuntu 22.04.
- Feed synchronisation: 181,187 NVT plugins loaded from the Greenbone Community Feed.
- Unauthenticated scan: soar system (192.168.1.112) scanned at network level — highest finding: 5.0 (Medium).
- Authenticated scan: ubuntu-self (192.168.1.113) scanned with SSH credentials — 442 findings including Critical 9.8 CVEs.
- Report verification: Detailed reports reviewed with CVE references, detection methods, affected software, and VendorFix remediation guidance.
- Key insight: Authenticated scanning reveals Critical 9.8 vulnerabilities completely invisible to unauthenticated network scans.

OpenVAS combined with Docker Compose provides a powerful, maintainable, and scalable open-source vulnerability management platform suitable for both small environments and enterprise deployments.